

04. Repaso de manejo de datos Y generación de gráficas.

Módulo 5. Inteligencia Artificial

**Jorge
Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Programa de la clase

- Practicar el uso del tidyverse con datos de tráfico y movilidad
- Aprender funciones nuevas de manejo de datos
- Aprender a hacer gráficas en ggplot

Operadores



¿Qué son los operadores?

Son símbolos que indican una operación a realizar sobre uno o más operandos (números, vectores, listas, data.frames, etc.).

Los más comunes son:

- `+` `-` `*` `/` : Suma, resta, multiplicación, división
- `^` : Potencia
- `%/%` : División entera (sin residuo)
- `%%` : Residuo (módulo)
- `:` : Secuencia
- `<`, `<=`, `>`, `>=`, `==` `!=` : Comparación (menor qué, menor o igual, mayor qué, mayor o igual, igual, diferente)
- `$` : Acceder a una columna de un data.frame
- `[]` : Acceder a un elemento de una lista
- `&`, `|` : “and” y “or”, para unir comparaciones en condiciones lógicas
- `<-` : Flechita de asignación
- `%*%` : Multiplicación matricial
- `%in%` : Pertenencia a algo
- `%>%` : Pipa del tidyverse
- `::` : Acceder a las funciones de un paquete
- `?` : Obtener ayuda (sobre una función).
- `??` : Obtener ayuda de internet.
- `~` : Fórmula

Función **unique()**

- La función **unique()** es la función que nos va a permitir visualizar los datos únicos de una columna o de un vector.
- Es útil para ubicar las categorías de una columna en una tabla.

Función *names()*

- La función **names()** nos permite conocer los nombres de una *tabla* (**tibble** o **data.frame**). O los nombres de los elementos de una *lista* (si es una lista con nombres). O los nombres de los elementos de un *vector-nombrado*.

Función *count()*

- La función **count()** nos permite contar el número de renglones en una tabla. Se puede usar en conjunto con **group_by()** para contar el número de renglones de un grupo dentro de una tabla.

Bucles *for*

- Los bucles **for** nos permiten iterar para los distintos valores de un vector de interés.
- La sintaxis es la siguiente:

```
for (opcion in opciones){  
    accion_1  
    accion_2  
    ...  
    accion_n  
}
```


Funciones propias

- En algunos casos, va a ser necesario que nosotros generemos nuestras funciones propias.
- Para ello, la sintaxis es:

```
nombre_funcion <- function(parametro_1,  
parametro_2, ..., parametro_n){
```

```
    ...cuerpo_funcion...
```

```
}
```


Función `case_when()`

- La función **`case_when()`** nos permite reclasificar los distintos valores dentro de un **`data.frame`**.
- La sintaxis es:

```
case_when(caso_1 ~ accion_1,  
           caso_2 ~ accion_2,  
           ...,  
           caso_3 ~ accion_3,  
           TRUE ~ accion_default)
```

Función *ifelse()*

- La función **ifelse()** nos permite programar acciones para cuando tenemos una condición lógica a probar.
- La sintaxis es:

```
ifelse(condicion_logica,  
       yes = accion_cuando_condicion_es_verdadera,  
       no  = accion_cuando_condicion_es_falsa)
```

pivot_wider() y pivot_longer()

¿Qué hace pivot_wider y pivot_longer?

Son funciones para reestructurar las tablas entre formato ancho y formato largo. pivot_longer pasa de formato ancho a largo, y pivot_wider pasa de formato largo a ancho.

Revisar:

<https://juvenalcampos.com/posts/2020-12-04-pivoteando-bases/>

pivot_wider() y pivot_longer()

¿Qué hace pivot_wider y pivot_longer?

Son funciones para reestructurar las tablas entre formato ancho y formato largo. `pivot_longer` pasa de formato ancho a largo, y `pivot_wider` pasa de formato largo a ancho.

Revisar:

<https://juvenalcampos.com/posts/2020-12-04-pivoteando-bases/>

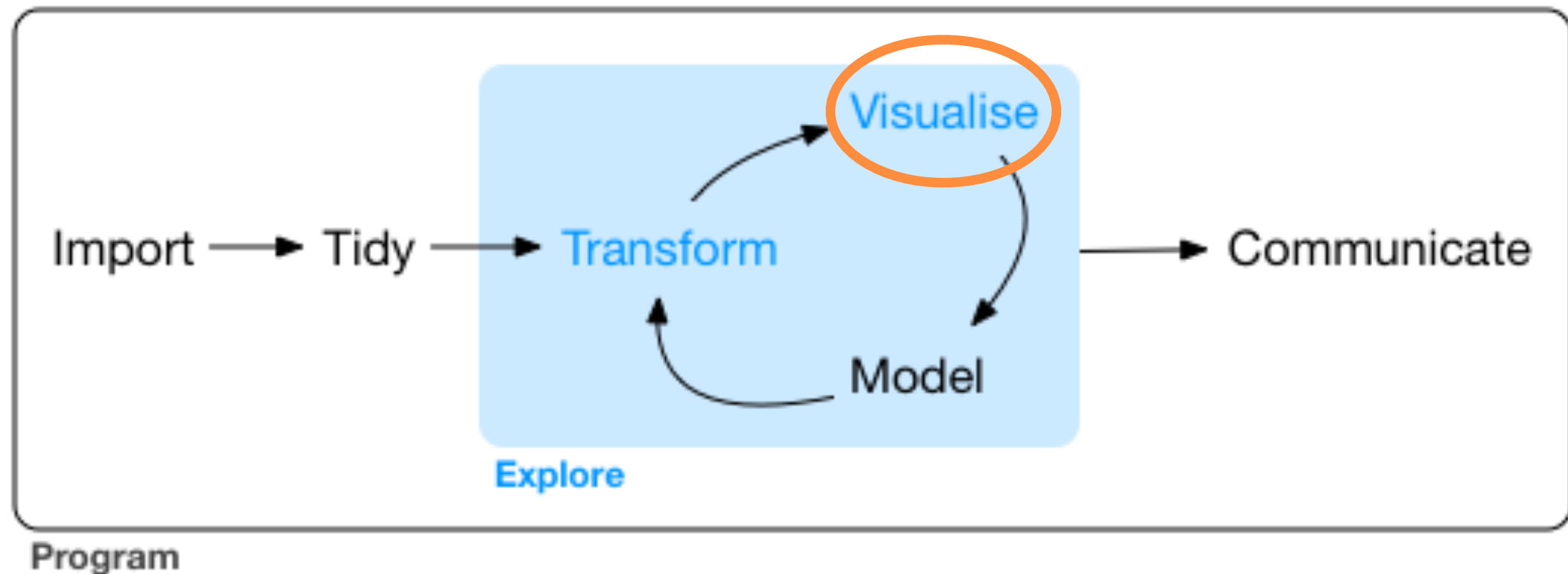
Visualización de datos en R



- * **El lenguaje de programación R provee librerías y herramientas para visualizar datos.** Las **ventajas** que ofrece realizar visualizaciones de R consisten en:
 - * **Alta personalización** de todos los elementos de nuestras gráficas.
 - * La posibilidad de trabajar **todo en un solo programa** (importación, exploración, limpieza, modelación, visualización y comunicación).
 - * La posibilidad de **generar visualizaciones interactivas** traduciendo código de R a HTML-CSS-JS y de generar aplicaciones web tipo shiny o documentos Markdown.

<https://www.data-to-viz.com/caveats.html>

Flujo de trabajo con datos



Hadley Wickham - R for data science
<https://r4ds.had.co.nz/explore-intro.html>

Paquetes de R para visualización



Los paquetes de R para visualización pueden incluir (mas no limitarse) a los siguientes:

Visualización estática:

- **base**: la librería base de R para hacer gráficas.
- **lattice**: librería que mejora las capacidades de r-base.
- **ggplot2**: la librería del tidyverse para hacer gráficas altamente personalizables.

Visualización interactiva:

- **plotly**, para gráficos interactivos básicos y compatible con ggplot2
- **Highcharter**, port de highcharts.js para R.
- **Amcharts**, visualización interactiva.
- **htmlwidgets**, familia de librerías para hacer visualización interactiva.
- **dygraphs**, para gráficas interactivas (especialmente Series de tiempo).
- **r2d3**, librería para hacer algunas gráficas de D3.js en R.

Mapas:

- **leaflet**: Mapas interactivos de leaflet.js
- También se pueden usar ggplot2, plotly o highcharter

Colores:

- **RColorBrewer**, paletas de colores.
- **viridis**, paletas de colores en R para personas con daltonismo.

Tablas:

- **DT**: Interface de la librería DataTable.js para R.
- **kableExtra**: Generar tablas con formato en HTML
- **formattable**: Tablas para presentación.
- **reactable**: Tablas con gráficas

Redes:

- **igraph**, para análisis y visualización básica de redes.
- **networkD3**, para redes interactivas y sankeys.



* Es la **librería del tidyverse más utilizada** para generar gráficas en R.

La programación de las gráficas requiere lo siguiente:

- 1) Un **objeto tibble/dataframe** (base de datos tabular) para darle la información.
- 2) **Iniciar el lienzo** con la función `ggplot()`
- 3) Definir los **elementos estéticos** (canales)
- 4) **Definir la(s) geometría(s)** o el tipo de gráfica a utilizar (marcas)
- 5) **Modificar elementos de la gráfica**, como etiquetas, formato de los ejes, sistema de coordenadas, límites de los valores, etc.
- 6) Opcional – **Definir temas y añadidos estéticos** para que la gráfica a se vea bonita.

1. Objeto tibble/dataframe



- 1) Un **objeto tibble/dataframe** (base de datos tabular) para darle la información.

```
> presion
# A tibble: 11 x 3
  systolicBloodPressure ageInYears weightInPounds
      <dbl>          <dbl>          <dbl>
1       132           52           173
2       143           59           184
3       153           67           194
4       162           73           211
5       154           64           196
6       168           74           220
7       137           54           188
8       149           61           188
9       159           65           207
10      128           46           167
11      166           72           217
```

2. Inicializar el lienzo

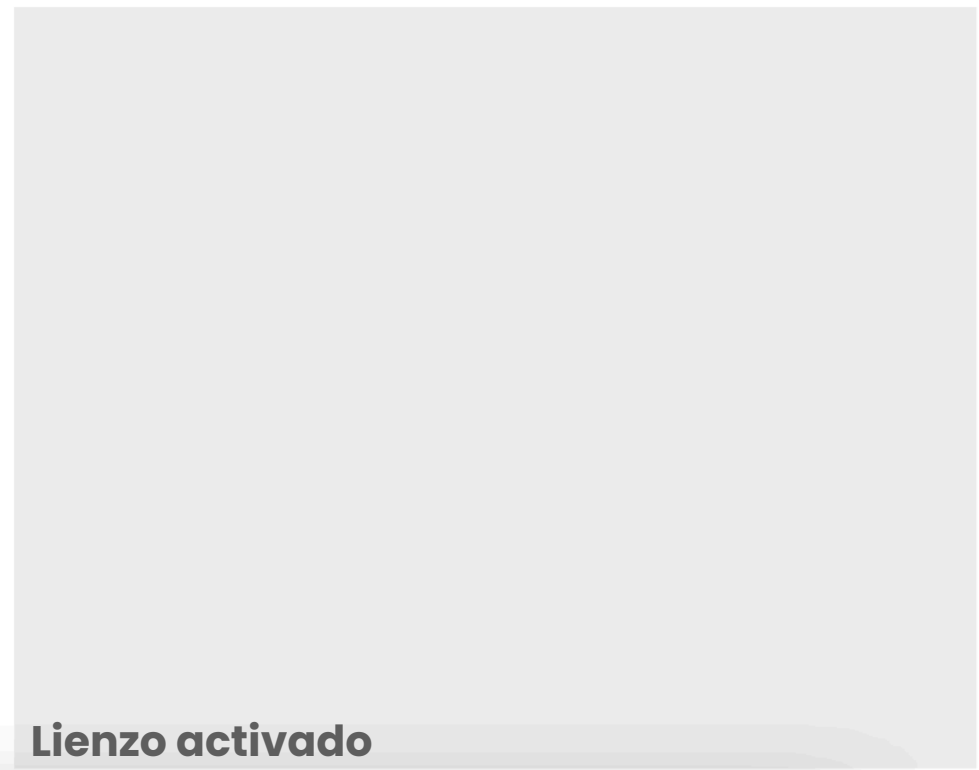


2) Inicializar el lienzo con la función ggplot()

- * Le pasamos el tibble del paso 1) a la función ggplot() para que se inicie el lienzo.
- * A partir de este punto las capas de la gráfica se van ligando con el símbolo de "+".
- * Dentro de ggplot() pueden irse incorporando los elementos estéticos.

```
presion %>%  
ggplot()
```

↑
Activar el lienzo



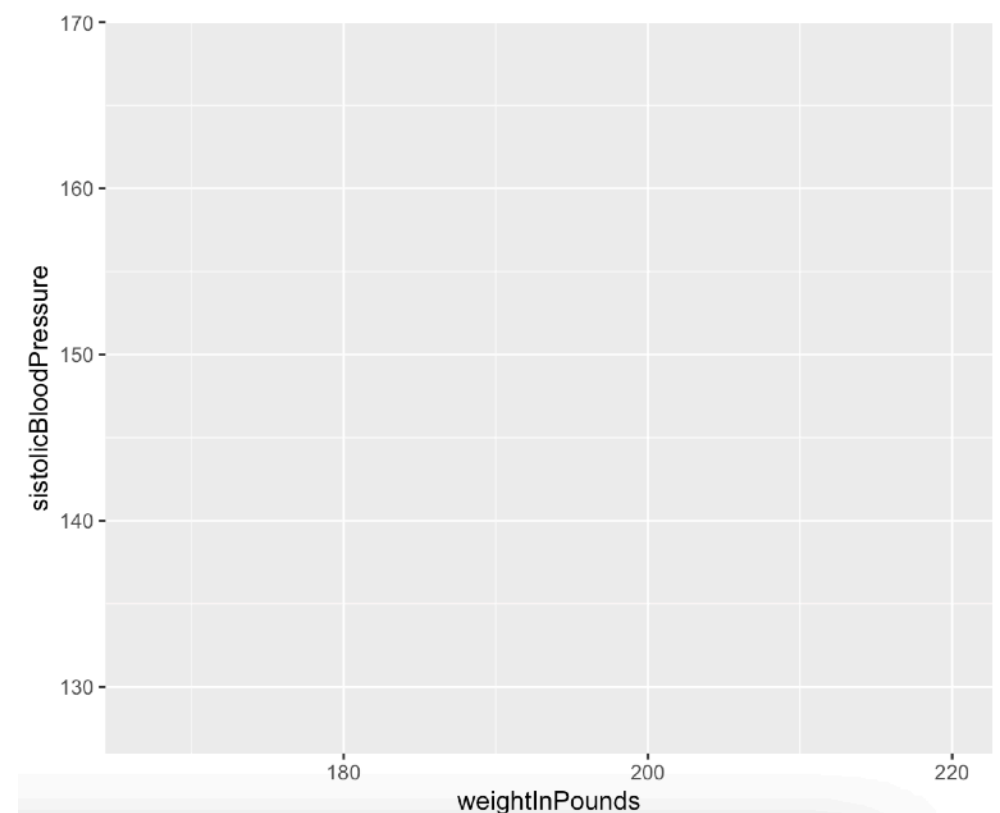
3. Definir los elementos estéticos



3) Definir los **elementos estéticos** (canales)

- * En este punto definimos las variables que van a definir la apariencia, no como la van a definir (esto se hace en un paso más adelante).
- * Los elementos estéticos a modificar son (pero no se limitan a) **la posición (coordenada x o y), el color, el relleno de una forma, el grosor de línea, el tamaño del punto, la transparencia, la forma que toma un punto o una línea, el grupo al que pertenece, etc.**

```
# Relación entre edad y presión sanguínea:  
presion %>%  
  ggplot(aes(y = systolicBloodPressure, # Posición eje Y  
             x = weightInPounds,        # Posición eje X  
             color = ageInYears         # Color de punto  
          ))
```



4. Definir geometrías o marcas



4) Definir la geometría o el tipo de gráfica a utilizar (marcas)

- * En este paso se decide qué **geometría** utilizar.
- * Cada geometría **requiere los elementos estéticos mínimos para funcionar** (por ejemplo, si vas a usar puntos en 2D se requiere que al menos definas la coordenada x y y).
- * **Las geometrías más comunes se pueden consultar acá:**
 - * <https://ggplot2.tidyverse.org/reference/>

4. Definir geometrías o marcas



* Algunos ejemplos:

Columnas y barras:



```
geom_bar() geom_col()  
stat_count()
```

Bins y mapas de calor:



```
geom_bin_2d()  
stat_bin_2d()
```

Líneas rectas:



```
geom_abline() geom_hline()  
geom_vline()
```



```
geom_path() geom_line()  
geom_step()
```

Mapas:



```
coord_sf() geom_sf()  
geom_sf_label()  
geom_sf_text() stat_sf()
```

Puntos:



```
geom_jitter()  
geom_point()
```

Densidades:



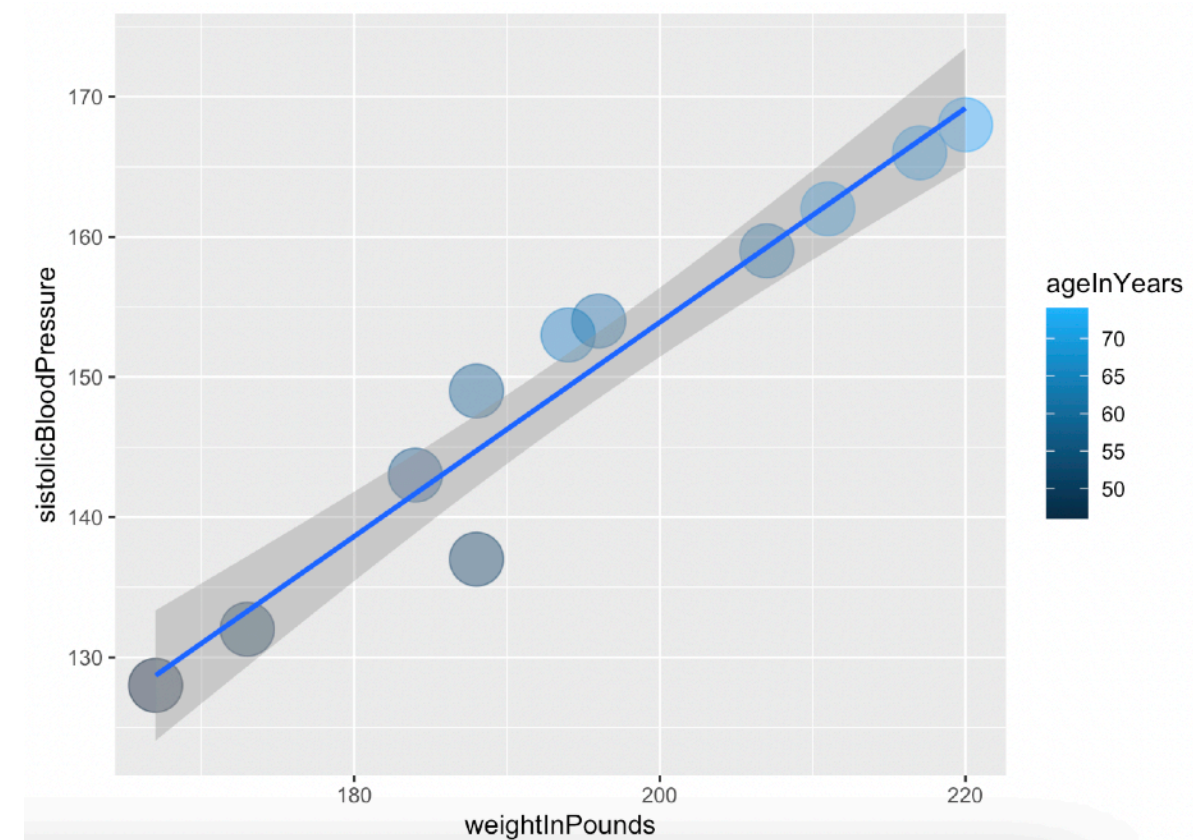
```
geom_boxplot()  
stat_boxplot()  
geom_density()  
stat_density()
```

4. Definir geometrías o marcas



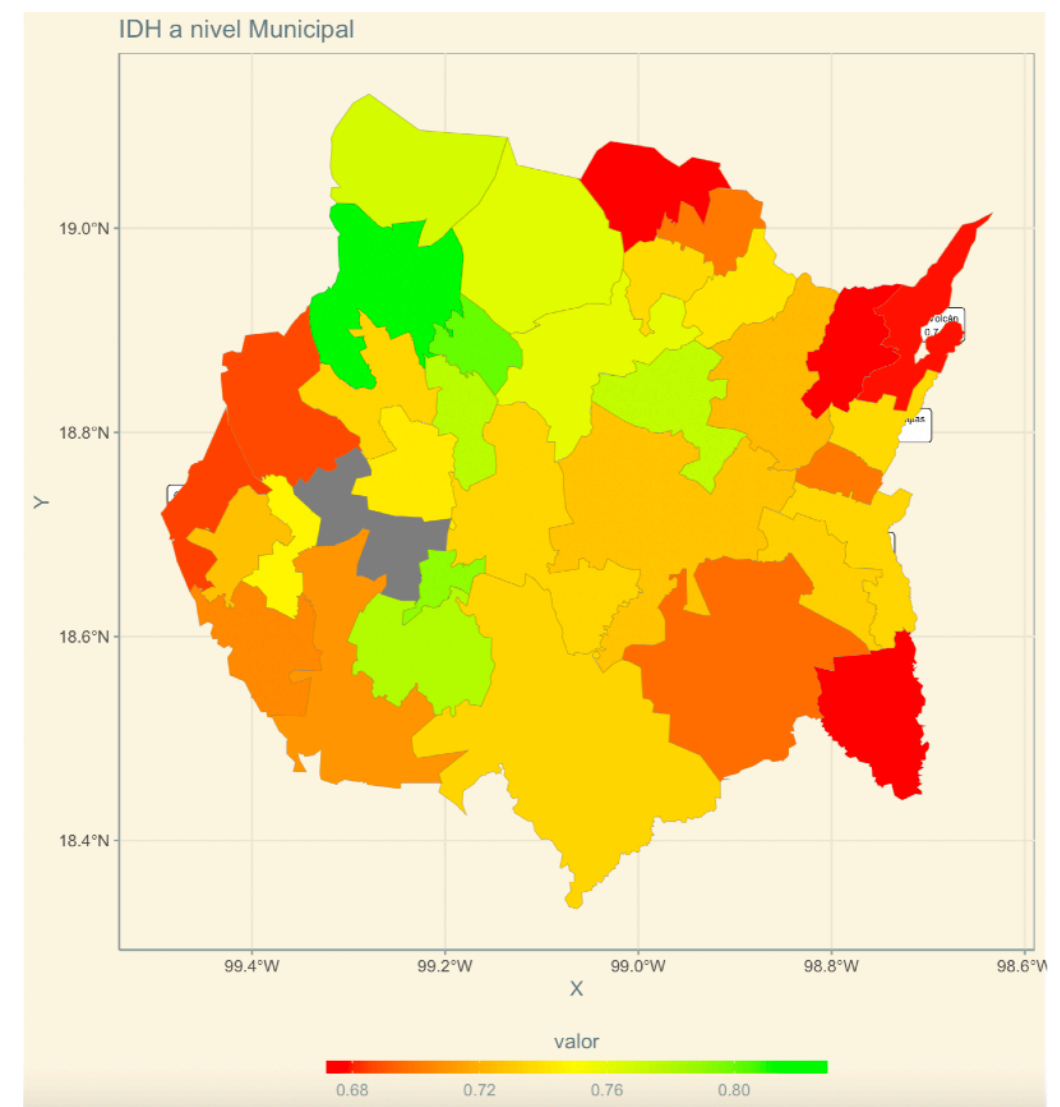
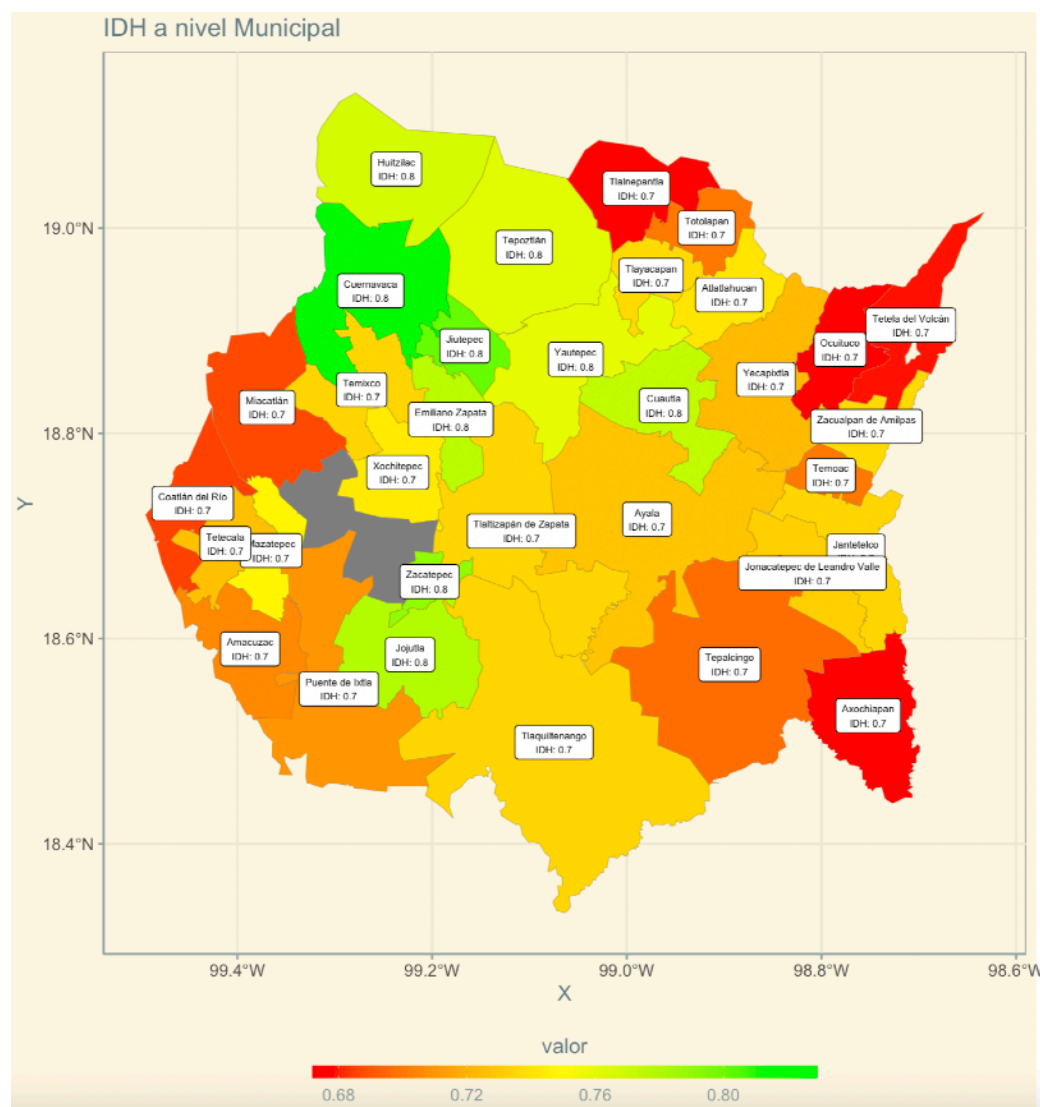
* Con código:

```
# Relación entre edad y presión sanguínea:
presion %>%
  ggplot(aes(y = systolicBloodPressure, # Posición eje Y
             x = weightInPounds,        # Posición eje X
             color = ageInYears         # Color de punto
           )) +
  geom_point(size = 10, alpha = 0.5) + # geometria de puntos
  geom_smooth(method = "lm")           # líneas de regresión
```



The ggplot2 logo, which is a hexagon containing a line plot with blue dots and the text "ggplot2" below it.

* **Nota:** El orden importa. La geometría que pongas primero va “abajo” y la que se ponga después va arriba.



5. Modificar elementos de la gráfica



5) Modificar elementos de la gráfica, como etiquetas, formato de los ejes, sistema de coordenadas, límites de los valores, etc.

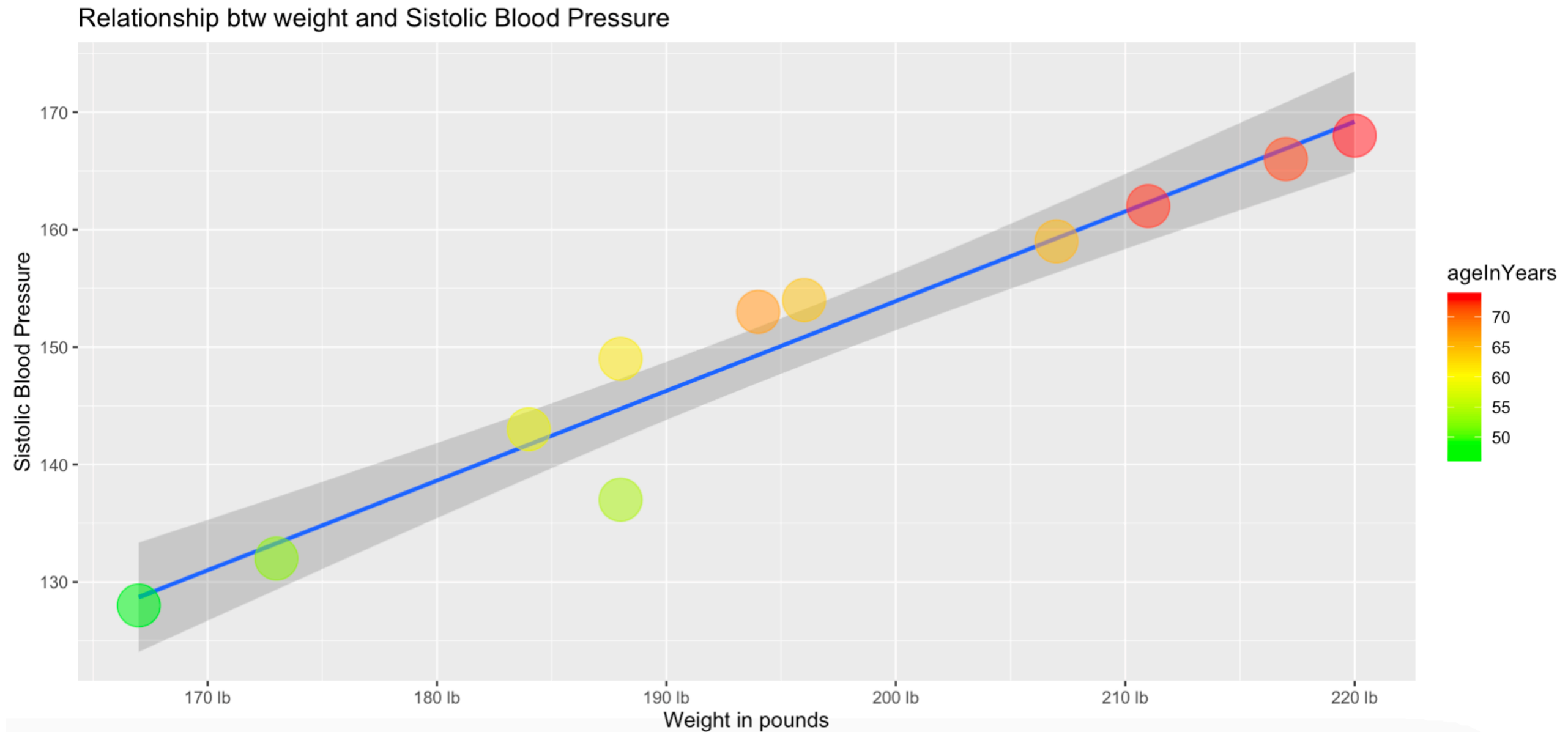
- * Para modificar los límites o los textos del eje x utilizamos las funciones `scale_x_*`()
- * Para modificar los colores, `scale_fill/color_*`()
- * Para añadir etiquetas, utilizamos la función `labs()` para modificar títulos, subtítulos, tags, pie de página, títulos de los ejes y de las leyendas.

5. Modificar elementos de la gráfica



```
# Relación entre edad y presión sanguínea:
presion %>%
  ggplot(aes(y = systolicBloodPressure, # Posición eje Y
             x = weightInPounds,        # Posición eje X
             color = ageInYears         # Color de punto
           )) +
  geom_smooth(method = "lm") +          # líneas de regresión
  geom_point(size = 10, alpha = 0.5) + # geometría de puntos
  # Modificamos el eje X (si es numerico)
  scale_x_continuous(
    breaks = seq(from = 100,
                  to = 230,
                  by = 10), # Cada cuando se ponen las marcas
    labels = scales::comma_format(suffix = " lb") # Sobre las etiquetas
  ) +
  # Modificamos los colores de las geoms que usan el aes "color"
  scale_color_gradientn(colors = c("green", "yellow", "red")) +
  # Etiquetas de la gráfica
  labs(title = "Relationship btw weight and Systolic Blood Pressure",
        x = "Weight in pounds",
        y = "Systolic Blood Pressure",
        caption = "Source: Textbook data"
  )
```

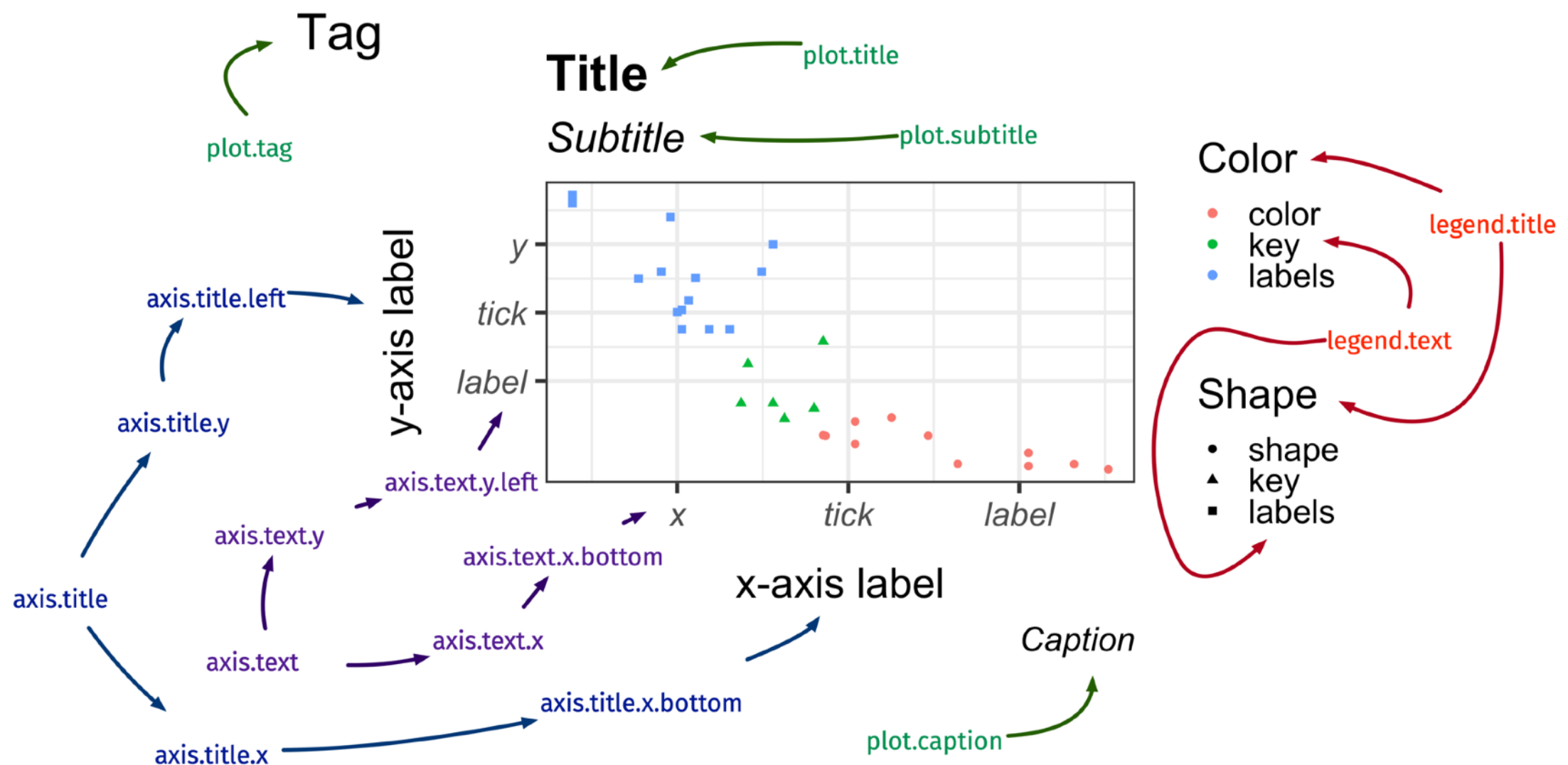
5. Modificar elementos de la gráfica



6.Temas: lo que se puede modificar



6) Opcional - **Definir temas y añadidos estéticos** para que la gráfica se vea bonita.



6.Temas: lo que se puede modificar



ggplot2 Theme Elements

`theme(element_name = element_function())`

- `element_text()`
- `element_line()`
- `element_rect()`
- `element_blank()`

Plot elements:

`plot.background`

`element_rect()`

`plot.title`

`element_text()`

`plot.margin`

`margin()`

Facetting elements:

`strip.background`

`element_rect()`

`panel.spacing`

`unit()`

`strip.text`

`element_text()`

Axis elements:

`axis.ticks`

`element_line()`

`axis.title`

`element_text()`

`axis.text`

`element_text()`

`axis.line`

`element_line()`

Legend elements:

`legend.margin`

`margin()`

`legend.title`

`element_text()`

`legend.key`

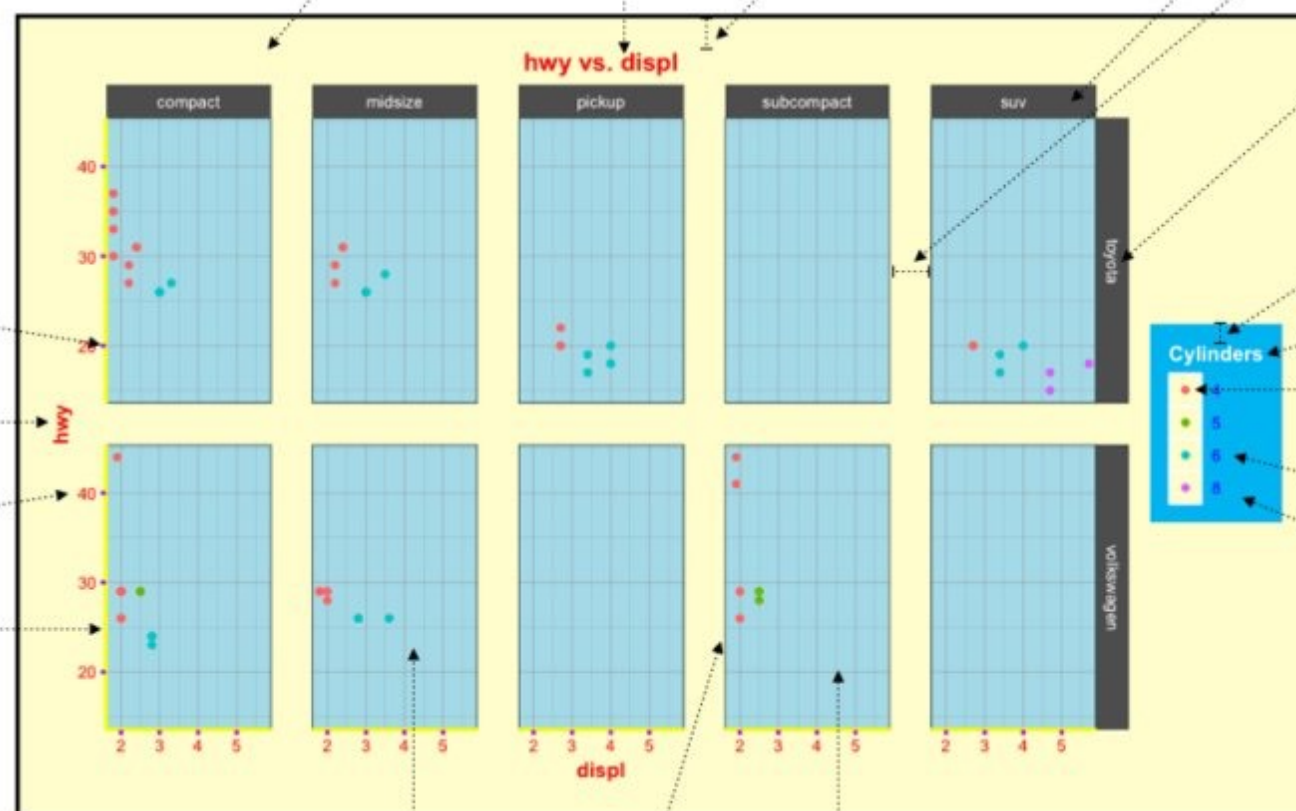
`element_rect()`

`legend.text`

`element_text()`

`legend.background`

`element_rect()`



`panel.background`

`element_rect()`

`panel.grid`

`element_line()`

`panel.border`

`element_rect(fill = NA)`

Panel elements:

henrywang.nl

Derived from "ggplot2: Elegant Graphics for Data Analysis"

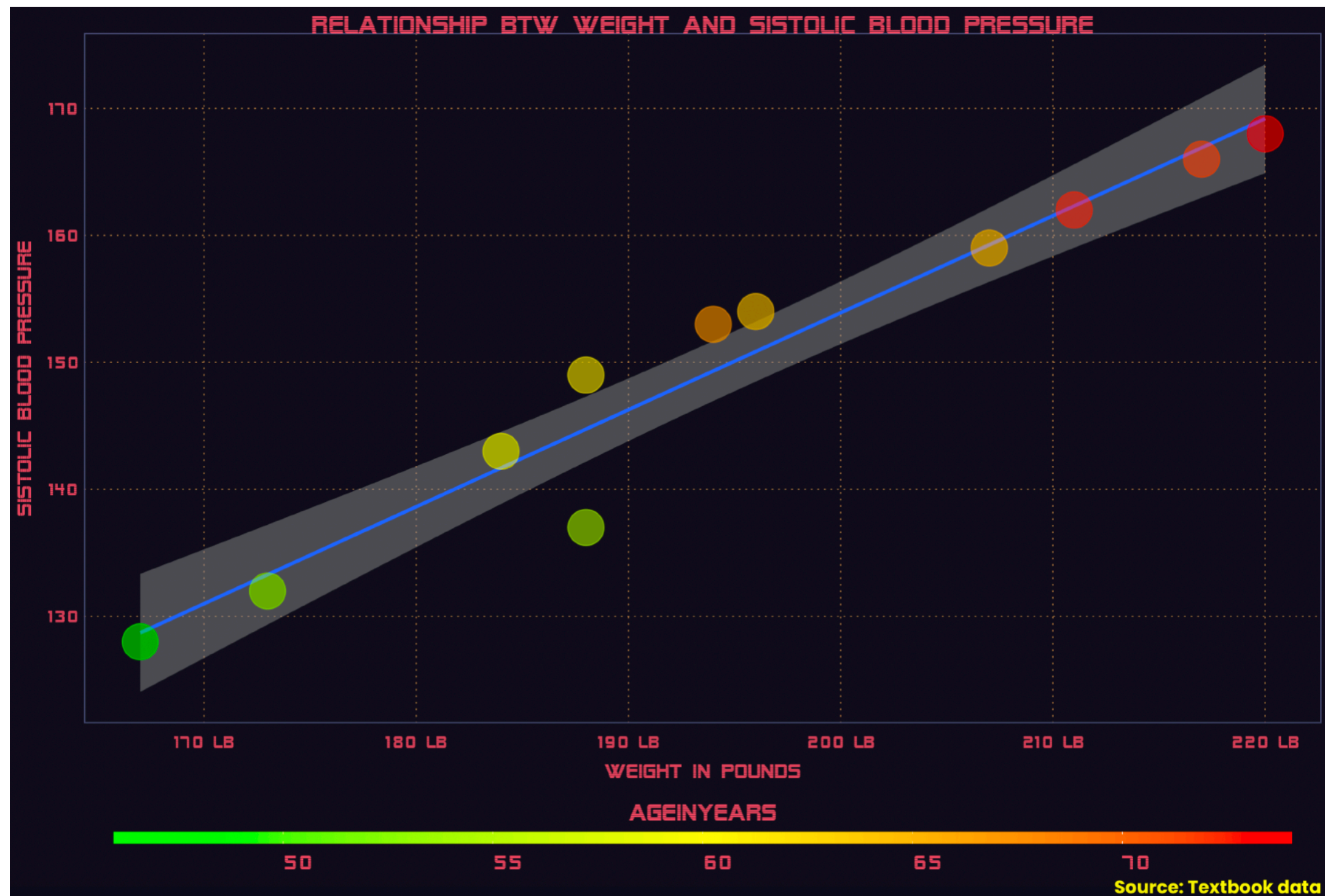
6.Temas: lo que se puede modificar



```
# Relación entre edad y presión sanguínea:
presion %>%
  ggplot(aes(y = systolicBloodPressure, # Posición eje Y
             x = weightInPounds,       # Posición eje X
             color = ageInYears        # Color de punto
           )) +
  geom_smooth(method = "lm") +          # líneas de regresión
  geom_point(size = 10, alpha = 0.5) + # geometría de puntos
  # Modificamos el eje X (si es numerico)
  scale_x_continuous(
    breaks = seq(from = 100,
                  to = 230,
                  by = 10), # Cada cuando se ponen las marcas
    labels = scales::comma_format(suffix = " lb") # Sobre las etiquetas
  ) +
  # Modificamos los colores de las geoms que usan el aes "color"
  scale_color_gradientn(colors = c("green", "yellow", "red")) +
  # Etiquetas de la gráfica
  labs(title = "Relationship btw weight and Systolic Blood Pressure",
        x = "Weight in pounds",
        y = "Systolic Blood Pressure",
        caption = "Source: Textbook data") +
  vapoRwave::new_retro() +
  guides(color = guide_colorbar(title.position = "top",
                                title.hjust = 0.5,
                                barwidth = 50,
                                barheight = 0.5)) +
  theme(axis.title.y = element_text(angle = 90),
        legend.position = "bottom",
        plot.caption = element_text(hjust = 1,
                                     family = "Poppins",
                                     face = "bold",
                                     color = "yellow" ))
```

Podemos utilizar temas
pre-programados:

6.Temas: lo que se puede modificar



Tips



1. **Define bien tus colores** y tus paletas de colores.
2. **Ten a la mano los iconos que utilices más a menudo**, y dedícale el tiempo necesario a tener tu carpeta de ícenos más utilizados (logos de tu empresa, de empresas con las que trabajes, partidos políticos, administraciones estatales, etc.)
3. Igualmente, **ten un repositorio de archivos que uses comúnmente**. Por ejemplo, para mapas yo tengo este repositorio de archivos: https://github.com/JuveCampos/Shapes_Resiliencia_CDMX_CIDE listos para crear mapas.
4. **Aprende como generar y subir páginas** en las cuales montar tus visualizaciones interactivas (Rmarkdown, Github Pages, shiny).
5. Trata de evitarlo, pero si no puedes, **hecha mano de la post-producción para editar tus gráficas** (por ejemplo, photoshop, illustrator o power point).

Más tips



1. Un buen ejercicio para empezar a practicar es el **replicar otras gráficas de otros programadores y usuarios**.
2. No te tienen que salir perfectas; recuerda que lo que te falte por hacer lo puedes corregir o añadir utilizando post-producción.
3. **Si estas en ceros en R**, te recomiendo que sepas: r-base, manejo de factores o variables categóricas, manejo de datos con el tidyverse, colores (rgb, hexadecimales) y pivoteo de bases (formato ancho y largo).
4. Si requieres **replicar una fuente** (tipo de letra) dada, usa la página <https://www.myfonts.com/WhatTheFont/> para guiarte y Google Fonts para descargar letras.

Datapasta (librería)



Brisbane area Partly cloudy. Light winds.	
3:30 pm, UV Index predicted to reach 11 [Extreme]	
Brisbane area Partly cloudy. Medium (50%) chance of showers, most likely in the late morning and afternoon. Light winds becoming easterly 15 to 20 km/h in the late afternoon then becoming light in the evening.	7 day Town Forecasts
3:30 pm, UV Index predicted to reach 11 [Extreme]	
Brisbane area Partly cloudy. Light winds.	

Location	Min	Max
 Brisbane	23	30
 Brisbane Airport	22	29
 Beaudesert	21	30
 Chermside	22	30
 Gatton	21	30
 Ipswich	21	31
 Logan Central	22	30
 Manly	23	28
 Mount Gravatt	22	29
 Oxley	22	31
 Redcliffe	23	28

Permite copiar y pegar tablas al script de R.

Gracias